

PRÁCTICA 2

Mario VILAR
20392724

David DíEZ
20392643

18 de octubre de 2023



UNIVERSITAT DE
BARCELONA

ÍNDICE

1	Introducción	2		<i>Ejercicio 10</i>	4
2	Preguntas	2		<i>Ejercicio 11</i>	4
	<i>Ejercicio 1</i>	2		<i>Ejercicio 12</i>	4
	<i>Ejercicio 2</i>	2		<i>Ejercicio 13</i>	4
	<i>Ejercicio 3</i>	2		<i>Ejercicio 14</i>	5
	<i>Ejercicio 4</i>	3		<i>Ejercicio 15</i>	5
	<i>Ejercicio 5</i>	3		<i>Figura 2.0</i>	5
	<i>Ejercicio 6</i>	3		<i>Ejercicio 16</i>	5
	<i>Ejercicio 7</i>	3		<i>Ejercicio 17</i>	5
	<i>Ejercicio 8</i>	3			
	<i>Ejercicio 9</i>	3			
			3	Conclusiones	5

INTRODUCCIÓN

La virtualización a nivel de SO (OS-level virtualization) es un paradigma en el que el sistema operativo permite crear espacios de usuarios aislados para los procesos que se ejecutan. Entre otras cosas, los contenedores permiten controlar:

1. El espacio de directorios/ficheros a los que pueden acceder;
2. el número máximo de CPUs que pueden utilizar;
3. la memoria RAM máxima que pueden utilizar;
4. el número máximo de procesos que se pueden ejecutar en un contenedor.

Los contenedores son administrados internamente por el sistema operativo. Docker es una herramienta de alto nivel muy utilizada actualmente que permite una fácil manipulación de contenedores con gran versatilidad.

PREGUNTAS

Ejercicio 1. *En el ejemplo anterior hello-world, ¿Qué error obtenemos al intentar eliminar la imagen de hello-world antes de eliminar el contenedor asociado a ella? ¿Cómo podríamos solucionar el problema de manera que podamos eliminar tanto la imagen como los contenedores asociados?*

Respuesta. El error que obtenemos al intentar eliminar la imagen es el siguiente:

```
Error response from daemon: conflict: unable to delete 9c7a54a9a43c
(must be forced) - image is being used by stopped container 01df8e427d01
```

Esencialmente esto quiere decir que la imagen se está utilizando y, por lo tanto, no es posible borrarla. Por otro lado, si intercambiamos el orden de ejecución entre las líneas `docker rmi 9c7a54a9a43c` y `docker rm 01df8e427d01`; es decir, ejecutamos primero `rm` (eliminamos el contenedor) y después `rmi` (eliminamos la imagen), no obtenemos errores y, efectivamente, tanto la imagen como el contenedor se eliminan. ✓

Ejercicio 2. *¿Qué otro comando se puede usar para eliminar al mismo tiempo todos los contenedores e imágenes no utilizados?*

Respuesta. Nos hemos ayudado de los comandos `man docker` y `docker help`, pero ninguno de ellos nos da lo que queremos. Buscando por la documentación de Docker nos encontramos con `prune`; en particular, la línea `docker system prune -a` poda imágenes (tanto las *danglings* como las no usadas), contenedores y *networks*. ✓

Ejercicio 3. *¿Qué versión de gcc tiene instalada en la máquina virtual y qué versión de gcc se instaló dentro del contenedor? Comente un beneficio de tener diferentes versiones de librerías instaladas dentro de un contenedor (en comparación con su máquina host). Es decir, ¿por qué puede resultar útil tener la versión “x” en la máquina host y la versión “y” dentro del contenedor?*

Demostración. La versión instalada en la máquina virtual es de 2021, 11.3.0. En cambio, la versión instalada en el contenedor es, a priori, la más reciente, 13.2.0 (de este mismo año). Nos puede ser útil por varias razones:

1. Nos permite ejecutar programas en versiones específicas (que, por ejemplo, solamente funcionan en versiones antiguas por cambios de sintaxis, librerías necesarias, dependencias)... En definitiva, evitamos los posibles conflictos.
2. Trabajar con diferentes versiones de las librerías dentro de contenedores sin afectar a la configuración del host. Esto nos permite hacer pruebas en un entorno aislado, sin afectar a otras aplicaciones fuera del contenedor ni interferir con las librerías instaladas en la máquina host.
3. Tener una versión específica dentro del contenedor nos permite exportar el proyecto entre máquinas de forma más sencilla sin preocuparnos por la compatibilidad. ✓

Ejercicio 4. Dentro del contenedor (*statistics*), intente instalar el editor “vi” usando `apt-get`. Comenta qué pasó y qué podría explicar eso.

Demostración. Intentamos instalar `vim` con estos dos comandos: `sudo apt-get install vim` y `apt-get install vim`. Pero en todos hemos obtenido problemas. No podemos ejecutar con `sudo` porque se este encuentra en `etc/sudoers`, y desde el contenedor no tenemos permisos para acceder a la carpeta `bin`. Como vimos en teoría, el contenedor deberá importar todas las dependencias necesarias para ejecutar satisfactoriamente nuestro código. El error sin el `sudo` consiste, en efecto, en un problema de permisos que nos intentábamos evitar usando `sudo` (`Permission denied`). Lo que podría explicar este comportamiento, entonces, no ejecutar este comando como usuario `appuser` y no administrador. ✓

Ejercicio 5. Intenta incluir la línea para instalar “vi” después de la línea “USER appuser”. ¿Has podido incluir las instrucciones para instalar “vi” después de la línea “USER appuser”? ¿Qué podría explicar eso? Al final, ¿qué instrucciones has incluido en el archivo `DockerFile` para instalar “vi”? Es decir, ¿que contiene tu archivo `DockerFile`?

Demostración. No hemos podido ejecutar `RUN apt-get install vim` ya que `USER appuser` parece cambiar el usuario en cuestión, de manera que `appuser` no tiene permisos para ejecutar la instrucción deseada: el error es el mismo que si intentamos hacer la instalación directamente en el contenedor, como en P4. Por tanto, tenemos que intercambiar el orden de ambas instrucciones. Con eso y añadiendo el flag `-y` (yes) a la instrucción, de manera que queda así: `RUN apt-get install -y vim`. Con todo, ya ha quedado instalado. ✓

Ejercicio 6. ¿Cómo se puede “despertar” un contenedor que ha sido “parado” (“STATUS” = “Exited”) de forma que se pueda introducir nuevas instrucciones dentro del mismo o utilizar lo que se quedó grabado en su interior? Escriba las instrucciones concretas para el contenedor *statistics* (después de haber salido de él).

Demostración. Primero de todo, hacemos un `exit`, de manera que el estado del contenedor cambia a `exited`. Consultamos la totalidad de los contenedores (tanto los activos como los no activos) con `docker ps -a`. Nos decidimos a despertar el que tiene imagen `statistics`, que tiene el `id` `1a6067f68852`, con `docker start 1a6067f68852`. Ahora, haciendo `docker ps` consultamos los contenedores activos y vemos, en efecto, que `1a6067f68852` es el único contenedor activo. En definitiva, la secuencia de instrucciones es la siguiente: primero `docker ps -a`, después `docker start 1a6067f68852` y por último `docker ps`. El flag `-a` nos permite ver *todos* los contenedores sin distinguir su estado. ✓

Ejercicio 7. ¿Cuántos procesos `fork-bomb` & se ejecutan dentro del contenedor después de ejecutarse `./fork-bomb`? ¿Cómo podemos comprobar este número?

Demostración. Ejecutando en el contenedor `sharp_thompson` los comandos requeridos, tenemos que `./fork-bomb &` nos da el número de líneas del fichero `Dockerfile` y, mientras tanto, ejecuta el bucle infinito que ya vimos en SOI. Una vez en `ctop`, podemos identificar claramente que el número de procesos ejecutados en el contenedor es 32, ya que en virtud de la tecnología `namespace` cada proceso tendrá su propio PID. ✓

Ejercicio 8. ¿Cuánta CPU está usando el contenedor al ejecutarse `./fork-bomb &`? Es decir, ¿utiliza 1, 2 o más CPUs? Ten en cuenta que en un ordenador con 4 CPUs, la CPU máxima que se puede usar es 400 % (podemos usar `ctop` para contestar a esta pregunta).

Demostración. Estamos usando una sola CPU, ya que al ejecutar el `docker run`, nos ha pedido hacerlo con los argumentos `--ulimit` en donde fijábamos el número de procesos y el de CPUs. El mismo `ctop` nos identifica que esta CPU está corriendo al 170 – 210 % en todo momento, hecho que es compatible con lo último que se nos dice en el enunciado. ✓

Ejercicio 9. ¿Podemos ejecutar comandos vía línea de comandos (en `bash`, por ejemplo, `ls` o `whoami`) dentro del contenedor después de que se ejecute `fork-bomb`? ¿Por qué `bash` da un mensaje de error?

Demostración. No podemos ejecutar el `ls` porque el programa anterior `fork-bomb` sigue en ejecución. De hecho, al intentarlo el error en cuestión menciona la instrucción `fork` que se está infinitamente ejecutando:

```
bash: fork: retry: Resource temporarily unavailable
```

En este sentido, el error va apareciendo periódicamente (cada segundo, más o menos), hasta que abortamos con `CTR+C`. Entonces, podemos intentar el comando con éxito. ✓

Ejercicio 10. *Intente ejecutar ambos servidores en dos terminales diferentes, sin utilizar ningún contenedor. ¿Qué es lo que sucede al intentar hacerlo?*

Demostración. No podemos ejecutar el mismo comando simultáneamente en dos terminales porque se escucha al mismo puerto¹. El error en particular es `bind() ha fallat, prova un altre port`. Si fijáramos el puerto *a posteriori* esto no sucedería. ✓

Ejercicio 11. *Describa brevemente, basándose en la documentación oficial de Docker, que es lo que permite hacer la opción `-p`. Indique también qué ocurre si no se utiliza la opción `-p` al ejecutar el contenedor (luego el servidor y luego el cliente). Proporcione el enlace de la página web oficial de Docker en la que basó su respuesta.*

Demostración. Primeramente hemos mirado en `docker run --help`, y hemos obtenido la información deseada: *Publish a container's port(s) to the host*. Si hacemos una breve búsqueda por internet (en particular, en la página de Docker), encontramos el mismo texto en el siguiente enlace: <https://docs.docker.com/engine/reference/commandline/run/>. ✓

Ejercicio 12. *¿Por qué actualmente se utilizan contenedores para ejecutar los servicios asociados con una aplicación grande en diferentes contenedores? ¿Qué ventajas aporta?*

Demostración. En particular para aplicaciones complejas y escalables se ha vuelto una práctica común debido a una serie de ventajas y beneficios que ofrecen:

1. *Portabilidad y consistencia:* Esto evita problemas relacionados con «funciona en mi máquina» al garantizar que la aplicación se ejecute igualmente en cualquier sistema compatible con contenedores.
2. *Aislamiento:* Básicamente, podemos hacer que cada servicio o componente de la aplicación se ejecute en su propio contenedor, los aísla entre ellos y dificulta una interacción problemática.
3. *Escalabilidad:* Puedes replicar contenedores en diferentes nodos o máquinas para manejar aumentos en la carga de trabajo, lo que mejora la eficiencia y la disponibilidad de la aplicación.
4. *Eficiencia de recursos:* Solo contienen lo estrictamente necesario para su funcionamiento, es decir, las bibliotecas y dependencias específicas de la aplicación, a diferencia de las MVs. ✓

Ejercicio 13. *¿Cuál fue la instrucción exacta que utilizó para ejecutar el contenedor de manera que pudiera acceder a la carpeta local desde el interior del contenedor?*

Demostración. El comando que hemos utilizado para ejecutar el contenedor para acceder a la carpeta local desde el interior ha sido `docker run -v /media/sf_shared-folder/practica4:/home/appuser/practica4/ gcc` donde `media/sf_shared-folder/practica4` es la ruta absoluta, `/home/appuser/practica4/` es la ruta del contenedor, `gcc` el nombre de la imagen y `-v` equivale a `--volume` que es el comando necesario para ejecutar lo que queremos. Los siguientes comandos:

```
docker run -ti -v /media/sf_shared-folder/practica4:/home/appuser/practica4/ gcc (dentro de la carpeta)
cd /home/appuser/practica4 (dentro del contenedor)
./mainRecc.exe 1 2207774 (dentro del contenedor)
```

¹ Parece que viene motivado por el hecho que al primer servidor ya se le asocia el descriptor de socket a su dirección local; al ejecutar el segundo, se intentará lo mismo con otra dirección distinta, sin éxito.

Son para poder ver que se ha ejecutado correctamente hemos tenido que acceder al interior del contenedor. ✓

Ejercicio 14. ¿Cuánto espacio en disco están utilizando los contenedores con y sin gcc. ¿Qué comando podemos utilizar para descubrir esta información?

Demostración. Se entiende que el contenedor sin gcc se refiere a aquel que utiliza la imagen de ubuntu e instala manualmente gcc en su interior. Recordad que tuvimos problemas con esto. Podemos utilizar el comando `docker ps -s -a`. Dicho comando muestra una lista de todos los contenedores junto con el espacio en disco utilizado por cada uno. Después de ejecutar los comandos de la pregunta 13 podemos ejecutar `exit` y `docker ps -s -a` y obtenemos que los contenedores que usan la imagen gcc ocupan unos 120B y los que no tienen gcc ocupan 80B. Lo relevante: en la memoria virtual: 77.8MB (ubuntu) y 1.38GB (gcc). ✓

Ejercicio 15. Muestra el contenido de tu DockerFile (el que usa Ubuntu como imagen base y tiene instalado Python 2 y vi).

Demostración. La primera parte del DockerFile la obtenemos de la foto que hay en el informe de la práctica (coincide con el DockerFile que encontramos en la carpeta de statistics). La segunda parte (derecha) la añadimos según el criterio que se nos

```
FROM gcc
RUN addgroup --gid 1000 appgroup
RUN useradd -r --uid 1000 -g appgroup appuser
RUN mkdir /home/appuser
RUN chown appuser:appgroup /home/appuser
COPY --chown=appuser:appgroup fitxers/* /home/appuser/
WORKDIR /home/appuser
USER appuser

FROM ubuntu:latest
FROM apt-get update
RUN apt-get install -y vim
RUN run apt-get install -y python2.7
CMD ["bash"]
```

pide: la imagen base es ubuntu e instalamos vim y python2.7 como se nos pide. ✓

Ejercicio 16. ¿Qué instrucciones utilizaste para

1. Generar la imagen.
2. Cargar la imagen generada a Docker Hub
3. Comprobar si Python 2 y “vi” se han instalados?

Demostración. Las instrucciones que hemos utilizado para cada caso han sido

1. `docker build -t myp2image`
2. Primero de todo ejecutamos `docker login`, lo cual nos pedirá el nombre de usuario y la contraseña de DockerHub. Luego ejecutamos `docker tag myp2image davidddiez/myp2image:tag`, seguido de `docker push davidddiez/myp2image:tag`
3. `python2.7 --version` y `vim --version` para comprobar que están instalados. ✓

Ejercicio 17. Muestra las instrucciones que utilizaste para descargar y ejecutar tu imagen myp2image desde Docker Hub. ¿Cuáles son las posibles ventajas de tener nuestras imágenes en repositorios como Docker Hub?

Demostración. Una vez eliminada la imagen y todas sus copias utilizando `docker system prune -a` podemos descargar la imagen desde DockerHub utilizando el comando `docker pull davidddiez/myp2image:tag`. Efectivamente, para ver que está todo bien podemos ejecutar la imagen utilizando `docker run -ti davidddiez/myp2image:tag` ✓

Hemos visto como la virtualización a nivel de SO y la tecnología de contenedores, como Docker, ofrecen ventajas significativas en términos de aislamiento de procesos, control de recursos, facilidad de administración y portabilidad. La práctica la hemos hecho íntegramente juntos, así que no nos hemos dividido mucho las tareas.